



Fig. 7: The Overview tool shows the entire image in a thumbnail view. The Pixel Region tool shows individual pixels from the small square region on the upper right tip, zoomed large enough to see the actual pixel values



Fig. 8: Grayscale and RGB images shown together

on the desktop and then display it using 'imtool':

```
>>B = imread('Penguins_grey.jpg');
>>imtool(B)
```

Fig. 5 shows the window that appears when using the image tool. The status text at the bottom of the main window shows the column/row location and the value of the pixel lying under the mouse cursor.

The Measure Distance tool under Tools tab is used to show the distance between the two selected points. Fig. 6 shows the use of this tool to measure distances between the beaks of different penguins.

The Overview tool under Tools tab shows the entire image in a thumbnail view. The Pixel Region tool shows individual pixels from the small square region on the upper right tip, zoomed large enough to see the actual pixel values. Fig. 7 shows the snapshot.

Multiple images can be displayed

within one figure using the subplot function. This function has three parameters within the brackets, where the first two parameters specify the number of rows and columns to divide the figure. The third parameter specifies which subdivision to use. For example, subplot(3,2,3) tells MATLAB to divide the figure into three rows and two columns and set the third cell as active. To display the images Penguins_RGB.jpg and Penguins_grey.jpg together in a single figure, you need to give the following commands:

```
>> A=imread('Penguins_grey.jpg');
>> B=imread('Penguins_RGB.jpg');
>>figure
>>subplot(1,2,1),imshow(A)
>>subplot(1,2,2),imshow(B)
```

The figure that appears on the screen after the third command is shown in Fig. 8.

Writing images. Images are written to the current directory using

the imwrite function, which has the following syntax:

```
imwrite(f, 'filename');
```

This command writes image data 'f' to the file specified by 'filename' in your current folder. The imwrite function supports most of the popular graphic file formats including GIF, HDF, JPEG or JPG, PBM, BMP, PGM, PNG, PNM, PPM and TIFF and so on.

The following example writes a 100x100 array of grayscale values to a PNG file named random.png in the current folder:

```
>> F = rand(100);
>>imwrite(F, 'random.png')
```

When you open the folder in which you are working, an image file named 'random.png' is created.

For JPEG images, the imwrite syntax applicable is:

```
imwrite(f, 'filename.jpg', 'quality', q)
where 'q' is an integer between 0 and 100.
```

It can be used to reduce the image size. The quality parameter is used as a trade-off between the resulting image's subjective quality and file size.

You can now read the image Penguins_grey.jpg saved in the current working folder and save it in JPG format with three different quality parameters: 75 (default), 10 (poor quality and small size) and 90 (high quality and large size):

```
F = imread('Penguins_grey.jpg');
imwrite(F,'Penguins_grey_75.jpg','quality',75);
imwrite(F,'Penguins_grey_10.jpg','quality',10);
imwrite(F,'Penguins_grey_90.jpg','quality',90);
```

Figs 9 through 11 show images for quality factors of 75, 10 and 90, respectively. You will note that that a low-quality image has a smaller size than a higher-quality image.

The imwrite syntax applicable to TIFF images has the format:

```
imwrite(g, 'filename.tif', 'compression',
'parameter', ..., 'resolution',
[colsrowsizes])
```